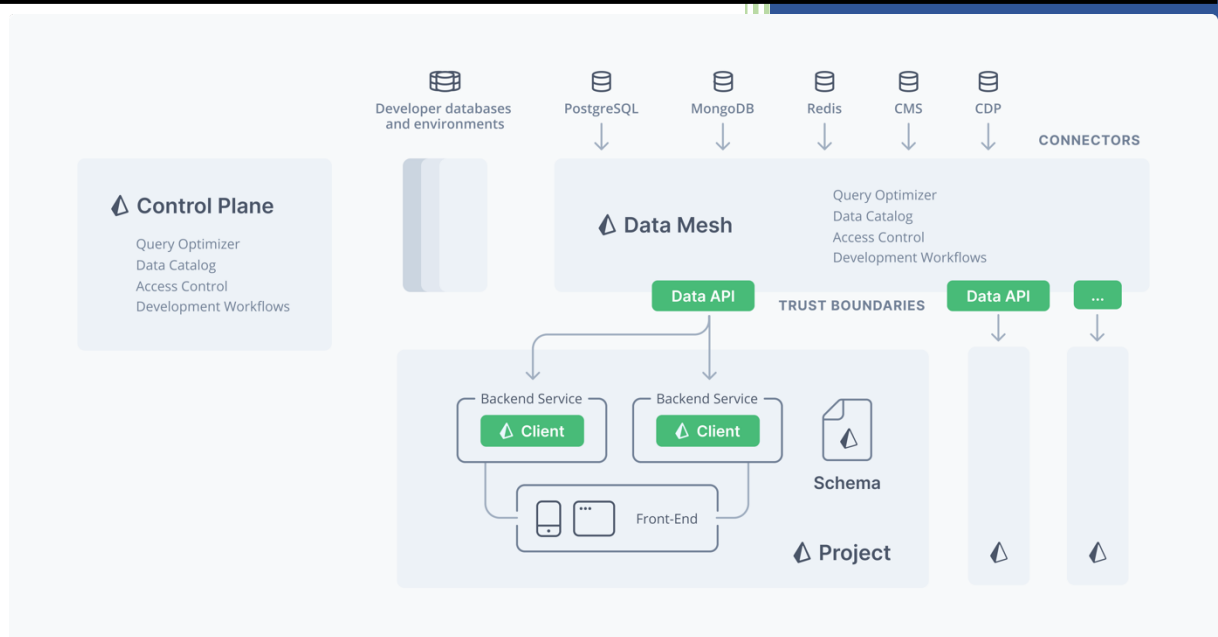


2024

Indications Backend avec Prisma



Dr H. KASSOGUE

hamidoukass@gmail.com

G4 Consulting

Sommaire

1. Installation et configuration du backend	2
2. Création des tables (schémas prisma, migrations)	3
3. Lancement du serveur backend	4
4. Contrôleurs et routes	5
4.1. Structuration du code	5
4.2. ProduitController et ProduitRouter	5
4.3. CommandeController et CommandeRouter	11
4.4. DetailsCommandeController et DetailsCommandeRouter	11
5. Références :	11

1. Installation et configuration du backend

On aura besoin d'un certain nombre d'outils pour travailler avec Prisma, dont :

- Prisma lui-même
- Express
- Dotenv
- Cors

On pourra adopter l'arborescence suivante pour notre projet:

```
|__ projectFolder
  |__ backend
  |__ frontend
```

Step 1. Initiation du backend

```
cd backend
npm init -y
```

Step 2. Installation des outils

Toujours dans backend

```
npm i express cors dotenv
npm i -D prisma
npm i @prisma/client
```

Step 3. Initialisation de prisma

```
npx prisma init
```

Cela crée un dossier prisma avec un fichier de configuration **.env**

```
|__ backend
  |__ node_modules
  |__ prisma
    +-- schema.prisma
    +-- .env
```

Step 4. Configuration de la base de données dans prisma

Dans **schema.prisma**, on écrira mysql à la place de postgresql (ligne 12) comme ceci

```
datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}
```

Et dans **.env**, on spécifiera les informations d'authentification à la base de données MySQL

```
DATABASE_URL="mysql://username:password@host:port/db_name"
```

Dans mon exemple, ce sera :

```
DATABASE_URL = "mysql://root:root@localhost:8889/l3gl-sotuba-eshop-react"
```

NB: La base de données devrait être déjà créée sur votre serveur MySQL.

2. Création des tables (schémas prisma, migrations)

Step 1. Créations des schémas prisma

Pour la création des schémas prisma, on pensera à installer l'extension Prisma sur votre IDE (VS Code par exemple) afin de bénéficier de la coloration syntaxique, les debug et autres. Dans le fichier `schema.prisma`, on ajoute le code :

```
model User {
  id          Int           @id @default(autoincrement())
  email       String        @unique
  username    String?       @unique
  firstname   String?
  lastname    String?
  avatar      String?
  password    String
  createdAt   DateTime       @default(now())
  role        String?
  Commandes   Commande[]
}

model Produit {
  id          Int           @id @default(autoincrement())
  nom         String
  categorie    String?
  prix        Decimal       @db.Decimal(9, 2)
  photo       String?
  description  String?       @db.Text
  createdAt   DateTime       @default(now())
  DetailsCommandes DetailsCommande[]
}

model Commande {
  id          Int           @id @default(autoincrement())
  num         String?       @unique
  userId      Int
  total       Decimal?      @db.Decimal(9, 2)
  status      String?
  createdAt   DateTime       @default(now())
  user        User           @relation(fields: [userId], references: [id],
onUpdate: Cascade, onDelete: NoAction)
  DetailsCommandes DetailsCommande[]
}

model DetailsCommande {
  id          Int           @id @default(autoincrement())
  commandeId  Int
  produitId   Int
  quantite    Int
  prix        Decimal       @db.Decimal(9, 2)
  createdAt   DateTime       @default(now())
  commande    Commande       @relation(fields: [commandeId], references: [id],
onUpdate: Cascade, onDelete: NoAction)
```

```

    produit      Produit      @relation(fields: [produitId], references: [id],
onUpdate: Cascade, onDelete: NoAction)
  }

```

Step 2. Migrations

Ensuite depuis le terminal, on effectue les migrations via les commandes :

```

npx prisma generate
npx prisma migrate dev

```

NB : la seconde commande aura pour objet de préparer le code SQL de migration et vous demandera de lui renseigner un nom pour la migration. On peut fournir ce nom depuis le lancement de la commande via

```

npx prisma migrate dev --name nomMigrationSouhaité

```

Une fois le fichier de migration créé (dans le sous-dossier migrations), Prisma se charge de la création des tables dans la base de données. Vous remarquerez une table additionnelle prisma_migrations, laquelle va permettre de suivre les migrations.

3. Lancement du serveur backend

Ajoutez cette ligne à `package.json` afin qu'on puisse plus tard importer et exporter des modules en utilisant la syntaxe ES6.

```

"type": "module",

```

On remarquera que dans `package.json`, une ligne indique que notre point d'entrée au serveur est `index.js`

```

"main": "index.js",

```

Ainsi, à la racine du backend, on crée un fichier `index.js` de lancement ayant pour contenu :

```

import express from 'express'
import cors from 'cors'
import dotenv from 'dotenv'

dotenv.config()

const app = express()

app.use(cors());
app.use(express.json());

const PORT = process.env.APP_PORT || 4000
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`)
})

```

Il faudra ajouter un paramètre `APP_PORT` au fichier `.env` pour le port qui sera utilisé pour le backend, par exemple

```

APP_PORT = 8000

```

Comme configuré, le port sera 4000 par défaut.

Pour lancer le serveur, on pourra faire juste

```
npm start
```

La combinaison de touche « CTRL + C » arrête le serveur. Cependant, à chaque modification des fichiers du serveur, il faudra qu'on redémarre le serveur. Nous allons utiliser **nodemon** afin qu'il s'occupe du redémarrage à chaque modification. Ainsi :

- On installe nodemon via (en mode développement)

```
npm i -D nodemon
```

- On ajoute à la clé "scripts" du fichier de dépendance **package.json**, les lignes

```
"dev": "nodemon",  
"start": "nodemon index.js",
```

4. Contrôleurs et routes

4.1. Structuration du code

Pour question de bonne structuration, on mettra les contrôleurs pour chaque table et les routes associés dans deux dossiers : **controllers** et **routes** respectivement. On les créera à la racine du backend comme ci-dessous :

```
|__ backend  
|__ controllers  
|__ node_modules  
|__ prisma  
|__ routers
```

4.2. ProduitController et ProduitRouter

Step 1. Dans le dossier **controllers**, on crée un fichier **ProduitController.js** ayant pour contenu :

```
import { PrismaClient } from "@prisma/client";  
  
export default class ProduitController {  
  prisma = new PrismaClient();  
  
  createProduit = async (req, res) => {  
    const data = req.body;  
    /**  
     * Vérifie si toutes les propriétés du corps de la requête ne sont pas vides  
     * @type {boolean}  
     */  
    if (Object.values(data).some((value) => !!value)) {  
      try {  
        const { id, nom } = data;  
        const produit = id ? await this.prisma.produit.findFirst({  
          where: {  
            OR: [  
              {  
                id: {
```

```

        equals: Number(id),
      },
    ],
    {
      nom: {
        equals: nom,
      },
    }
  ]
},
}) : await this.prisma.produit.findFirst({
  where: {
    nom: nom,
  },
});
if (produit) {
  res
    .status(200)
    .json({ status: false, msg: "Ce produit existe déjà" });
} else {
  const newProduit = await this.prisma.produit.create({
    data: data,
  });
  res.status(200).json({
    status: true,
    msg: "Produit créé avec succès",
  });
}
} catch (error) {
  console.error("Erreur lors de la requête à la base de données:", error);
  res
    .status(500)
    .json({ status: false, message: "Erreur interne du serveur" });
}
} else {
  res.status(404).json({ status: false, message: "Aucune donnée reçue" });
}
};

updateProduit = async (req, res) => {
  const data = req.body;
  const { id } = req.params;
  /**
   * Vérifie si toutes les propriétés du corps de la requête ne sont pas vides
   * @type {boolean}
   */
  if (Object.values(data).some((value) => !!value)) {
    try {
      const produit = await this.prisma.produit.findUnique({
        where: {
          id: Number(id),

```

```

    },
  });
  if (produit) {
    const updatedProduit = await this.prisma.produit.update({
      where: {
        id: Number(id),
      },
      data: data
    });
    res.status(200).json({
      status: true,
      msg: "Produit modifié avec succès",
    });
  } else {
    res
      .status(404)
      .json({ status: false, msg: "Ce produit n'existe pas" });
  }
} catch (error) {
  console.error("Erreur lors de la requête à la base de données:", error);
  res
    .status(500)
    .json({ status: false, message: "Erreur interne du serveur" });
}
} else {
  res.status(404).json({ status: false, message: "Aucune donnée reçue" });
}
};

findProduit = async (req, res) => {
  const data = req.params;
  /**
   * Vérifie si toutes les propriétés du corps de la requête ne sont pas vides
   * @type {boolean}
   */
  if (Object.values(data).some((value) => !!value)) {
    try {
      const { element } = data;
      const produit = await this.prisma.produit.findFirst({
        where: {
          OR: [
            {
              id: {
                equals: Number(element),
              },
            },
            {
              nom: {
                equals: element,
              },
            },
          ],
        },
      });
    }
  }
};

```



```

    ],
  },
});
if(produit) {
  res.status(200).json({
    status: true,
    data: produit,
  });
} else {
  res
    .status(404)
    .json({ status: false, msg: "Ce produit n'existe pas" });
}
} catch (error) {
  console.error("Erreur lors de la requête à la base de données:", error);
  res
    .status(500)
    .json({ status: false, message: "Erreur interne du serveur" });
}
} else {
  res.status(404).json({ status: false, message: "Aucune donnée reçue" });
}
};

getProduits = async (req, res) => {
  try {
    const produits = await this.prisma.produit.findMany({
      select: {
        id: true,
        nom: true,
        prix: true,
        categorie: true,
        photo: true,
        description: true,
      },
    });
    res.status(200).json({
      status: true,
      data: produits,
    });
  } catch (error) {
    console.error("Erreur lors de la requête à la base de données:", error);
    res.status(500).json({ status: false, message: "Erreur interne du serveur"
});
  }
};

deleteProduct = async (req, res) => {
  const data = req.params;
  /**
   * Vérifie si toutes les propriétés du corps de la requête ne sont pas vides

```

```

    * @type {boolean}
    */
    if (Object.values(data).some((value) => !!value)) {
      const { id } = data;
      try {
        const produit = await this.prisma.produit.findUnique({
          where: {
            id: Number(id)
          },
        });
        if(produit) {
          await this.prisma.produit.delete({
            where: {
              id: Number(id)
            },
          });
        };
        res
          .status(200)
          .json({ status: true, msg: "Produit supprimé avec succès" });
      } else {
        res
          .status(404)
          .json({ status: false, msg: "Ce produit n'existe pas" });
      }
    } catch (error) {
      console.error("Erreur lors de la requête à la base de données:", error);
      res.status(500).json({ status: false, message: "Erreur interne du serveur"
    });
  }
} else {
  res.status(404).json({ status: false, message: "Aucune donnée reçue" });
}
};
}

```

Step 2. Ensuite, dans le dossier **routes**, on crée un fichier **ProduitRoutes.js** ayant pour contenu :

```

import express from "express"
import ProduitController from "../controllers/ProduitController.js"

const produitRouter = express.Router()
const produitController = new ProduitController()

// Routes pour les produits

// Rechercher un produit par un critère spécifique (par exemple, nom, catégorie)
produitRouter.get("/produits/:element", produitController.findProduit)

// Récupérer tous les produits
produitRouter.get("/produits", produitController.getProduits)

// Créer un nouveau produit

```

```

produitRouter.post("/produits", produitController.createProduit)

// Mettre à jour un produit existant
produitRouter.put("/produits/:id", produitController.updateProduit)

// Supprimer un produit
produitRouter.delete("/produits/:id", produitController.deleteProduct)

export default produitRouter

```

Step 3. Il ne reste plus qu'à utiliser les routes de produits dans `index.js` comme suit :

```

import produitRouter from './routers/produitRoutes.js'
.
.
.
app.use(express.json())
app.use(produitRouter)
.
.
.

```

Les cinq routes pour produits sont maintenant opérationnelles. On peut utiliser Postman ou encore l'extension « REST Client » sous l'IDE VS Code pour les tester. A propos le code ci-dessous est un exemple d'utilisation de « REST Client » à travers un fichier `request.rest` placé à la racine du backend.

```

POST http://localhost:8000/produits
Content-Type: application/json

{
  "id": 1,
  "nom": "Appareil photo",
  "categorie": "Electro",
  "prix": 35000,
  "photo": "product-1.jpg",
  "description": "L'appareil photo Canon... Lorem ipsum dolor sit amet
consectetur adipisicing elit.
}

###
GET http://localhost:8000/produits

###
GET http://localhost:8000/produits/1

###
PUT http://localhost:8000/produits/1
Content-Type: application/json

```

```
{
  "nom": "Appareil photo Canon",
  "categorie": "Electronique",
  "prix": 35000,
  "photo": "product-1.jpg",
  "description": "L'appareil photo Canon... Lorem ipsum dolor sit amet
consectetur adipisicing elit. Veritatis, nostrum ipsum ea iusto porro quae dolore
est cumque minima voluptas assumenda modi ex inventore repellendus! Quae nobis eius
at nostrum!"
}

###
DELETE http://localhost:8000/produits/1
```

NB : le serveur doit être démarré pour que les tests marchent.

4.3. CommandeController et CommandeRouter

Step 1. Dans le dossier **controllers**, créez un fichier **CommandeController.js** en se basant sur le code de **ProduitController.js**.

Step 2. Dans le dossier **routes**, créez un fichier **CommandeRoutes.js** en se basant sur le code de **ProduitRoutes.js**.

Step 3. Dans **index.js**, utilisez les routes de commande comme procédés pour produit. Et testez-les sur Postman ou REST Client.

4.4. DetailsCommandeController et DetailsCommandeRouter

En se basant sur les codes précédants, réalisez les étapes 1-3 pour la table **detailsCommande**.

Voir le dépôt distant <https://github.com/HKassogue/simple-eshop-backend> pour un exemple de code complet.

5. Références :

- Coder media ; Tutorial CRUD Realtime dengan Node JS, Express, MySQL, React JS (Prisma + SWR) ; disponible sur <https://www.youtube.com/watch?v=OQspok-swzY> ; Consulté le 23/06/2024.
- Prisma docs ; Get started on prisma ; Disponible sur <https://www.prisma.io/docs/getting-started> ; Consulté le 23/06/2024.